

	<i>Programmation Système</i> C-shell & Processus	Année universitaire : 2025/2026
---	--	---

Objectifs :
• Comprendre les principes des processus

Exercice 1: La différence entre un processus et une fenêtre

- Affichez la liste de vos processus, par la commande `ps`
- Lancer `gedit` depuis l'interpréteur de commande : `gedit &`. Remarquez le numéro écrit dans le terminal, après le nombre entre crochet. C'est un numéro (unique) qui sert à identifier le processus : c'est son PID (processus identifier).
- Observez de nouveau la liste de vos processus. Il y en a un de plus que précédemment : le `gedit` qui vient d'être lancé, repérable à son PID justement.
- Tapez un court texte dans `gedit` et le sauver dans un fichier `toto`
- Ouvrir une nouvelle fenêtre
- Observer la liste des processus (avec `ps`) : il n'y a pas de nouveau processus.
- Lancez de nouveau `gedit` depuis l'interpréteur de commande

RQ : Dans le premier cas, les 2 fenêtres sont gérées par un même processus. Celui-ci connaît donc les informations de chacune des 2 fenêtres. Dans le deuxième cas, on a lancé un nouveau processus. Son espace de données est disjoint du processus précédent : les exécutions de ces 2 processus sont indépendantes.

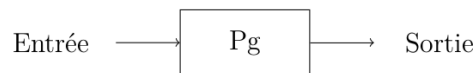
Exercice 2: Utilisation de kill

- Dans un terminal, lancez la commande : `less toto`
- Ouvrez un deuxième terminal, dans lequel vous lancez la commande `ps -u` pour afficher tous les processus que vous avez lancé. Vous devriez trouver le processus `less` que nous venons de lancer
- Faites maintenant `kill pid`, où `pid` est le numéro du processus `less`
- Refaites la manipulation précédente, en utilisant `kill -9` à la place de `kill`. Comparez le message donné par `less` avec le précédant.
- `kill -9 -1` : permet de tuer tous les processus de l'utilisateur courant. C'est donc un moyen efficace mais très violent de terminer sa session ! (aucune sauvegarde possible du travail en cours)
- Lancez un programme (script c-shell), après quelques secondes d'exécution, tapez `Ctrl + C` bien été interrompu

RQ : la commande *kill* « demande » au processus de se terminer, mais le processus à qui on envoie ce signal peut l'ignorer.

Exercice 3: Redirection des Entrées Sorties

Un processus peut effectuer une transformation sur de l'information : il lit de l'information en entrée, effectue un traitement, et donne de l'information en sortie. Le processus lit l'information sur l'entrée standard, écrit de l'information sur la sortie standard, sachant que l'entrée standard est généralement affectée au clavier, et la sortie standard est généralement affectée à l'écran.



- > : redirige stdout vers le fichier indiqué après >
- >> : comme >, mais si le fichier existait, on ajoute à la suite (mode append)
- 2> : redirection de stderr vers le fichier indiqué
- 1> : comme >
- < : redirection d'entrée, le fichier indiqué après < remplace stdin

Le shell permet de réaliser cette redirection de manière très simple. Quand on désire exécuter la commande *commande* en redirigeant son entrée sur le fichier *nom-de-fichier-entrée*, il suffit de taper : *commande > nom-de-fichier-sortie*. Si on désire rediriger l'entrée et la sortie, il suffit de taper : *commande < nom-de-fichier-entrée > nom-de-fichier-sortie*

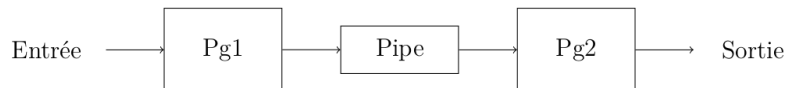
- a- Créer deux fichiers *fich1* et *fich2*
- b- Regardez par *less* (ou *more*) le contenu de ces deux fichiers
- c- Maintenant, réexécutez *cat* de façon à mettre la concaténation des contenus de *fich1* et *fich2* dans un fichier de nom quelconque, par exemple *resu1*, et vérifiez en suite à l'aide de *more*, que le fichier *resu1* contient bien le résultat espéré
- d- Utilisez la commande *sort* et la redirection d'entrée/sortie pour trier un fichier.
- e- La commande *wc* (word count) permet de compter non seulement les mots, mais également les caractères et les lignes du texte qui lui est fourni en entrée standard :
 - wc -c* : compte les caractères
 - wc -w* : compte les mots
 - wc -l* : compte les lignesDans un terminal tapez : *wc -w*. Tapez après quelques mots, et terminez par *Ctrl + D* juste après une entrée pour indiquer la fin des données. Après cela, *wc* doit vous indiquer le nombre de mots que vous avez tapés.
- f- Utilisez *wc -w* pour compter le nombre de mots du fichier *toto*
- g- Comptez le nombre de mots du fichier *toto*, en envoyant le résultat dans le fichier *resu2*. Vérifiez à l'aide de *more* que *resu2* contient bien le résultat espéré.

Exercice 4: Les pipes

Il arrive très souvent que l'on désire faire coopérer deux processus dans une chaîne, c'est-à-dire, de manière à ce que le second effectue son travail sur l'information produite par le premier



Pour effectuer cette coopération on utilise une espèce de tuyau (pipe) à information entre les deux processus.



Dans le shell, on utilise la syntaxe suivante pour demander l'établissement d'un pipe: *commande1 | commande2*

- a- La commande *sort -n -k i* prend un tableau en entrée et exécute le tri de ce tableau selon la valeur croissante de la i-ème colonne. Utilisez cette commande (avec *ls*) pour afficher le contenu d'un répertoire, trié par taille.
- b- Utilisez *ls* et *wc* pour compter le nombre d'objets qui se trouvent sur votre répertoire courant.

En générale, il est possible d'enchaîner les commandes afin qu'une commande travaille sur le résultat de la commande précédente... Il s'agit en fait de brancher le *stdout* d'une commande sur le *stdin* de la commande suivante... et ainsi de suite... Les « pipes » utilisent souvent des filtres, c'est à dire des commandes qui modifient, découpent, réorganisent ou simplifient le flux entrant (c'est à dire ce qui arrive sur *stdin*). Les filtres sont les commandes suivantes : *grep*, *cut*, *tr*, *sort*, *uniq*, *wc*, *tail*, *head*, *sed*... On utilise aussi *wc* (compte les mots, lignes et autres caractères), et *more* ou *less* (pager, qui permettent le découpage lisible)

Exercice 5: Tâches de fond

- a- Lancez un terminal, et dans ce terminal, lancez la commande *gedit*. Vous pouvez utiliser *gedit*, mais vous n'avez plus la main dans le terminal.
- b- Dans un terminal, lancez la commande *gedit*. Revenez au terminal sans fermer la fenêtre de *gedit*, puis faites *Ctrl + z*. Lancez une autre commande dans le terminal, par exemple, *ls*. Essayez maintenant de vous servir de la fenêtre *gedit* : plus rien n'a l'air de marcher, *gedit* est suspendu. Revenez dans le terminal et faites *bg*. Vous gardez la main dans le terminal (i.e. vous pouvez exécuter d'autres commandes), mais *gedit* reprend vie. Essayez maintenant la commande *fg* : on est revenu dans la situation initiale.

- c- Lancez la commande *gedit &*. Dans le même terminal, lancez une autre commande comme *ls* : elle devrait s'exécuter normalement. Essayez d'utiliser la fenêtre *gedit* que vous venez de lancer, puis fermez cette fenêtre.

Exercice 6: Alias

- a- Essayez de lancer la commande suivante dans un terminal : *alias toto=ls*. À partir de maintenant, quand vous entrerez la commande *toto*, elle se comportera comme *ls*. Fermez maintenant ce terminal puis ouvrez-en un autre. L'effet de la commande a disparu.
- b- Il serait pénible de devoir entrer à la main la configuration du shell à chaque lancement de terminal, c'est pourquoi bash prévoit un mécanisme d'initialisation : le fichier *.bashrc*, situé à la racine de votre compte, sera lu à chaque démarrage de shell. Vous pouvez vous amuser à y mettre des alias mais ne gardez pas ce genre de chose dans votre configuration. Exemple : *gedit ~/.bashrc* et ajouter des alias, après tapez *source ~/.bashrc*.

Exercice 7: Variable PATH

- a- Dans un terminal, lancez la commande *echo "\$PATH"*, elle affiche le contenu de la variable *\$PATH*. Elle devrait commencer par quelque chose comme */usr/local/bin:/usr/bin:/bin*. Quand on entre une commande comme *firefox*, le shell va tester la présence d'un exécutable *firefox* dans chacun des répertoires. Comme */usr/local/bin/firefox* n'existe pas (on peut le vérifier avec *ls*), le shell passe au suivant, */usr/bin/firefox*, qui existe, donc le shell exécute ce fichier. Pour savoir où se trouve une commande, on peut utiliser la commande *type*, comme ceci : *type nom-de-commande*.
- b- Dans le fichier *.bashrc* ajoutez *dir1=/usr/include*. La référence à la valeur d'une variable se fait par *\$nom-de-variable*. Exemple : *echo \$dir1* affichera */usr/include*.